# DELTAVERSE integration spec: agents, identity, payments, and post-quantum constitution

## Executive summary

This spec ties **AgenticPlace, mindX, BANKON, DAIO/BONAFIDE, openBDK BDK6, and Parsec** into one coherent stack with a clear separation of concerns: **EVM is the constitutional layer, Algorand is the identity and post-quantum receipt layer, BDK6 is the audit replica, and x402 over USDC-on-Algorand is the universal payment rail**. Seven decisions drive the whole design:

1. **Deploy the 9-contract DAIO/BONAFIDE suite to Polygon PoS first**, mirror it to an AggLayer-connected CDK chain only after AggLayer v0.4 fast interop ships. Polygon zkEVM is sunsetting in 2026 and is excluded.
2. **Issue BANKON AlgoIDNFT as an ARC-72 soulbound smart-contract NFT** whose `arc72_transferFrom` reverts unconditionally, anchored to a `did:algo:mainnet:app:<APP_ID>:<SUBJECT>` DID document.
3. **Adopt x402 v2 with the official `scheme_exact_algo` AVM scheme**, integrate against the `@x402-avm/*` SDK family (production-ready since late 2025, Algorand Foundation–endorsed via GoPlausible), settle in USDC ASA 31566704 with optional fee-payer abstraction. github GitHub
4. **Bridge Algorand → EVM via state proofs through the SNARK pathway**, not a hand-rolled Solidity Falcon-1024 verifier; verifying 640 Falcon sigs + SumHash512 paths natively on EVM is on the order of 2–4 billion gas, infeasible without a Groth16 wrapper.
5. **Expose a four-level transaction finality enum in Parsec** — `pending → consensus_final → state_proof_final → cross_chain_final` — so users see when their history becomes post-quantum anchored.
6. **Port ARC-52 (BIP32-Ed25519, Khovratovich-Law) into Parsec's native language** rather than wrapping `AlgoNode/go-algorand-hd-wallet` via FFI/WASM; the cryptographic surface is < 600 LOC and the Foundation reference libs (TS/Kotlin/Swift) are pre-audit but architecturally clean.
7. **Use Foundry for the EVM suite and AlgoKit 3.0 + Algopy for the Algorand side in parallel**, but accept that Algorand has no in-process invariant fuzzer equivalent to `revm` and budget 10–50× longer integration test runs.

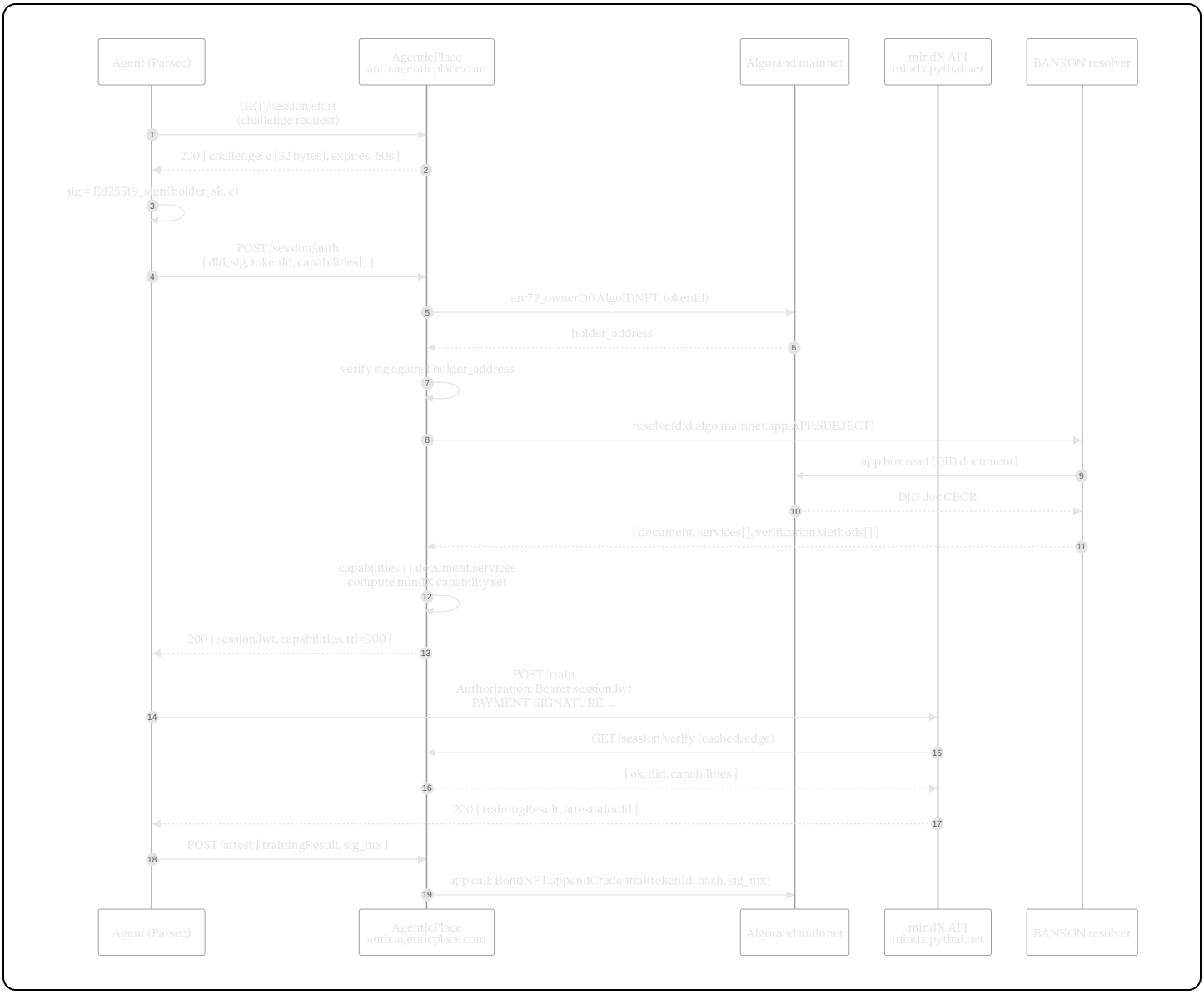
The rest of this document is the production specification for those decisions, including code sketches in Solidity, Algopy, TypeScript, Python, and PHP.

## 1. AgenticPlace ↔ mindX ↔ BANKON integration spine

The three services form a triangle: **AgenticPlace** is the marketplace and protocol gateway, **mindX** is the gamma-brainwave training capability, and **BANKON** is the identity authority. Every agent-facing call into mindX must carry a verifiable BANKON credential and resolve to a payable session. The cleanest factoring is:

- **AgenticPlace** issues *session tokens* (short-lived JWTs) bound to a verified DID and a paid x402 invoice.
- **mindX** trusts only AgenticPlace-issued session tokens or, optionally, raw DID-auth presentations for power users. It never sees Algorand directly.
- **BANKON** is queried *once per session* by AgenticPlace's auth service, never on every mindX call.

## 1.1 Authentication flow




The session JWT is RFC 9068 (`sub = did:algo:...`, `cnf = thumbprint of holder pubkey`, `aud = mindx.pythai.net`, `cap = ["gamma:train","gamma:profile","attest:v1"]`). mindX validates it against AgenticPlace's JWKS endpoint and caches the verification at the edge to keep p99 below 50 ms.

## 1.2 Credential-to-capability mapping

The DID document's `service` array drives capability negotiation. Map types one-to-one to mindX endpoints:

| DID service `type` | mindX capability | Default rate (per session) |
|---|---|---|
| `MindXAgentEndpoint` | `gamma:train` | 60 calls / 15 min |
| `BonafideIdentityNFT` | `attest:v1` | unlimited reads, 10 writes |
| `DAIOMembership` | `gamma:profile:dao` | 600 calls / 15 min |
| `GeniusBearer` | `gamma:train:advanced` | unlimited |

mindX rejects any capability that is not present in both the JWT `cap` claim *and* the DID document services. AgenticPlace is the issuer; mindX is the enforcer; BANKON is the registry. This factoring lets mindX scale horizontally without holding Algorand client state.

### 1.3 REST + MCP contract for mindX

mindX exposes both shapes because AI agents prefer MCP tool calls and traditional clients prefer REST. The MCP server simply wraps the REST endpoints.

```yaml
```

```yaml
# mindx OpenAPI (excerpt)
/train:
  post:
    security: [{ bearer: [] }]
    headers:
      X-Idempotency-Key: { schema: { type: string }, required: true }
      PAYMENT-SIGNATURE: { schema: { type: string }, required: false }
    requestBody:
      application/json:
        schema:
          type: object
          required: [profile, durationMs, targetBand]
          properties:
            profile:     { type: string, enum: [novice, focus, deep, recovery] }
            durationMs:  { type: integer, minimum: 30000, maximum: 1800000 }
            targetBand:  { type: object, properties: { minHz: {type:number}, maxHz:{
            biofeedback: { type: object } # optional
    responses:
      "200":
        application/json:
          schema:
            type: object
            properties:
              attestationId: { type: string, format: uuid }
              sessionHash:   { type: string, description: "sha256 of session payload
              metrics:
                type: object
                properties:
                  meanGamma:   { type: number }
                  coherence:   { type: number, minimum: 0, maximum: 1 }
                  bandPower:   { type: array, items: { type: number } }
              signature:
                type: object
                description: "Ed25519 signature by mindX over (sessionHash || did ||
                properties: { sig: {type:string}, kid:{type:string}, alg:{type:strin
```

The MCP wrapper exposes `mindx.train`, `mindx.profile`, and `mindx.attest` as `paidTool` definitions per Vercel's `x402-mcp` pattern, so MCP clients invoke them with one line and the 402 dance is invisible. IQ.wiki WorkOS

### 1.4 Attestation back-propagation

Every successful training session must be *attested back to BANKON* so reputation accumulates

on-chain. The attestation payload is `keccak256(sessionHash || did || timestamp || metricsCommitment)`, signed by mindX's Ed25519 key and submitted as an inner application call to a `BondNFT` Algorand app:

```python
# Algopy: BondNFT credential append (excerpt)
@arc4.abimethod
def append_credential(self, token_id: arc4.UInt256,
                      attestation: arc4.StaticArray[arc4.Byte, 32],
                      sig: arc4.StaticArray[arc4.Byte, 64],
                      kid: arc4.Address) -> None:
    # Only accepted attestors (mindX, peer DAIOs) can write.
    assert self.attestors[kid].native, "unknown attestor"
    pk = kid.bytes  # Algorand address == ed25519 pubkey
    msg = self.token_did[token_id].native + attestation.bytes
    assert op.ed25519verify_bare(msg, sig.bytes, pk)
    # Append to box-stored credential log; bound to MAX_CREDENTIALS.
    log_box = self.credentials.get(token_id, default=Bytes(b""))
    self.credentials[token_id] = log_box + attestation.bytes
```

Rate limiting is enforced at AgenticPlace, not mindX, by counting attestations per `(did, hour)` against the session JWT's `cap` budget. mindX trusts the JWT and never sees the rate counter.

---

## 2. DAIO deployment onto blockchain

The 9-contract BONAFIDE/DAIO suite — Genius, BonaToken, Tabularium, Fides, SponsioPactum, Censura, Senatus, Tessera, plus a Constitution contract — is the missing piece. The recommended deployment is a **dual-layer architecture**: constitutional logic on EVM, identity and PQ-anchored receipts on Algorand, audit replication on BDK6.

### 2.1 Target chain selection

Polygon PoS wins as the launch target on six axes. It is mature, EVM-equivalent so the Solidity ports as-is, has ~$0.001 transaction cost, runs a ~110-validator PoS quorum, has no AggLayer multisig dependency, and is not deprecating. **Polygon zkEVM is excluded** because it is on a 2026 sunset path. **Polygon CDK as a sovereign chain** is excluded for launch because it inherits Polygon's 2-of-3 timelock plus 6-of-8 security council over the CDK contracts on Ethereum, layered on top of your own sequencer responsibility — a meaningful trust addition that adds little value for a constitutional DAO at launch TVL. **Self-hosted BDK6 (1+3 BFT)** is excluded as the constitutional layer because three validators with 2-of-3 quorum is acceptable for audit logs but unacceptable for a public DAO holding capital; a coordinated compromise of two keys ends

the game. BDK6 instead becomes a **shadow append-only audit replica**, receiving every constitutional event for organizational record-keeping. AMINA Bank

| Decision | Choice | Rationale |
|---|---|---|
| Constitutional logic | **Polygon PoS (chainId 137)** | Cheap, decentralized, EVM-equivalent, no sunset |
| Future cross-chain mirror | AggLayer CDK chain (Katana or own) | Wait for v0.4 fast interop |
| Identity + receipts | **Algorand mainnet** | BANKON already there; ARC-72 soulbound; state proofs |
| Audit replica | **openBDK BDK6** | Append-only, no funds, no liveness criticality |
| Bridge direction | Algorand → EVM (state proof) + EVM → Algorand (oracle) | Trustless one way, federated other way |

### 2.2 Contract topology and immutability discipline

Eight of the nine contracts deploy as **immutable** (no proxy). The only upgradeable contract is **Tabularium** (the reputation accumulator), which uses a UUPS proxy whose `_authorizeUpgrade` is gated by a Timelock + 4-of-7 Safe. This matches the constitutional thesis: rules of the game cannot be edited from inside the game, but the bookkeeping ledger evolves under audited governance.

```
Genius          (immutable)  - role-bearing membership ERC-721
BonaToken       (immutable)  - non-transferable governance unit
Tabularium      (UUPS proxy) - reputation accumulator
Fides           (immutable)  - reputation rules / decay parameters
SponsioPactum   (immutable)  - bonded commitments / slashing
Censura         (immutable)  - dispute resolution / role revocation
Senatus         (immutable)  - proposal engine + vote aggregator
Tessera         (immutable)  - cross-chain anchor / DID linkage
Constitution    (immutable)  - genesis hash, top-level rule registry
```

### 2.3 Foundry deploy script and ordering

```solidity
```

```
// script/Deploy.s.sol
contract Deploy is Script {
    bytes32 constant SALT = keccak256("DAIO_BONAFIDE_v1");

    function run() external {
        vm.startBroadcast();

        Constitution con  = new Constitution{salt: SALT}(GENESIS_HASH);
        Genius       gen  = new Genius{salt: SALT}(address(con));
        BonaToken    bona = new BonaToken{salt: SALT}(address(con));
        Fides        fid  = new Fides{salt: SALT}(address(con));

        // Tabularium: UUPS
        TabImpl impl = new TabImpl();
        ERC1967Proxy tab = new ERC1967Proxy(
            address(impl),
            abi.encodeCall(TabImpl.initialize, (address(gen), address(fid)))
        );

        SponsioPactum sp  = new SponsioPactum{salt: SALT}(address(tab));
        Censura       cen = new Censura{salt: SALT}(address(gen), address(tab));
        Senatus       sen = new Senatus{salt: SALT}(address(tab), address(cen));
        Tessera       tes = new Tessera{salt: SALT}(address(con), ALGORAND_LIGHTCLIE

        // Wiring
        gen.setSenatus(address(sen));
        bona.setSenatus(address(sen));
        TabImpl(address(tab)).setSenatus(address(sen));
        con.seal(address(sen), address(cen), address(tes));

        vm.stopBroadcast();
    }
}
```

Run with `forge script script/Deploy.s.sol --broadcast --slow --verify --resume`. The `--slow` flag eliminates receipt-polling races between dependent transactions. Foundry

### 2.4 Testing strategy

Foundry invariants are the highest-leverage tests for constitutional rules. Each invariant maps to a clause of the constitution:

```solidity
```

```
contract ConstitutionalInvariants is StdInvariant, Test {
    function setUp() external {
        deployer = new Deploy();
        deployer.run();
        targetContract(address(handler)); // realistic action handler
    }

    /// CL-1: total reputation never exceeds total minted BonaToken supply.
    function invariant_ReputationBoundedBySupply() public view {
        assertLe(tab.totalReputation(), bona.totalSupply());
    }

    /// CL-2: no member simultaneously holds Genius and Censura admin.
    function invariant_RoleNonOverlap() public view {
        for (uint256 i = 0; i < members.length; i++) {
            assertFalse(
                gen.isGenius(members[i]) && cen.isAdmin(members[i])
            );
        }
    }

    /// CL-3: treasury conservation.
    function invariant_TreasuryConservation() public view {
        assertEq(treasury.balance + handler.totalOutflows(),
                 handler.totalInflows());
    }
}
```

Set `runs = 1024, depth = 128` for an overnight run; `show_metrics = true` to inspect handler call distribution and discard rate.

### 2.5 Governance bootstrap

The chicken-and-egg problem: how do the first members get Genius status before there is a DAIO to grant it? Solution is a **time-locked genesis ceremony**:

1. **Genesis ceremony (T-0 to T+7d):** The deployer EOA mints the initial Genius set (≤ 7 founders) directly via a one-shot `genesisMint(address[] founders)` function. After T+7d a `sealGenesis()` call permanently disables `genesisMint` by flipping an immutable storage bit.
2. **Founder→DAO handover (T+7d to T+30d):** Founders pass a single proposal transferring all admin roles from the deployer EOA to the Senatus + Timelock. Deployer EOA is then

irrevocably removed from `RoleRegistry`.

3. **Citizen onboarding (T+30d onward):** Open membership via `applyForCitizenship(did, attestations[])`. Each application is voted on by Genius holders weighted by Tabularium reputation.

THRUST token-gating intersects DAIO at the **validator and citizen levels**, not the Genius level: a validator must hold THRUST (BDK6 token) and be a citizen (BonaToken holder), but does not need to be a Genius. This three-tier separation prevents any single token from dominating governance.

### 2.6 Migration without losing constitutional state

Testnet → mainnet migration must preserve the constitutional state hash without preserving testnet votes. The pattern:

```solidity
// On testnet: snapshot
bytes32 stateRoot = constitution.exportStateRoot();
// stateRoot commits to: founders, Fides parameters, role wirings — NOT historical v

// On mainnet: re-anchor
constructor(bytes32 _genesisHash, bytes32 _testnetStateRoot) {
    GENESIS_HASH = _genesisHash;
    TESTNET_ANCESTOR = _testnetStateRoot; // informational; not consulted in voting
}
```

Historical votes from testnet are explicitly discarded; only the founder set, Fides parameters, and Tessera cross-chain anchor configuration carry over. This avoids the trap of importing stale reputation that was farmed on a faucet-funded testnet.

---

## 3. x402 Algorand payment rail through Parsec

x402 v2 is the production AI-agent payment standard as of April 2026, governed by the x402 Foundation (Coinbase + Cloudflare), with **Algorand as a first-class network** thanks to the merged `specs/schemes/exact/scheme_exact_algo.md` and the `@x402-avm/*` reference SDK family from GoPlausible (Algorand Foundation-endorsed). GitHub

### 3.1 Wire format and headers

Three V2 headers carry the protocol: `PAYMENT-REQUIRED` on the 402 challenge, `PAYMENT-SIGNATURE` on the retry, and `PAYMENT-RESPONSE` on the settled 200. All three are

base64-encoded JSON. Backward-compat aliases `X-PAYMENT` and `X-PAYMENT-RESPONSE` from V1 are still recognized inbound but the AgenticPlace edge emits V2 only. x402

The Algorand `exact` scheme uses a **2-transaction atomic group** for fee abstraction: an unsigned `pay` from the facilitator to itself (covering pooled fees) plus a client-signed `axfer` of µUSDC to the resource server. The facilitator signs the fee leg, broadcasts the group, and returns the txid. The client never holds ALGO; only USDC ASA opt-in (0.1 ALGO MBR) is required. github GitHub

```json
// PAYMENT-REQUIRED (decoded base64), AgenticPlace charging $0.01 per inference call
{
  "x402Version": 2,
  "accepts": [{
    "scheme": "exact",
    "network": "algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=",
    "amount": "10000",
    "asset": "31566704",
    "payTo": "AGENTICPLACE_TREASURY_ADDR_BASE32...",
    "maxTimeoutSeconds": 60,
    "extra": { "feePayer": "FACILITATOR_ADDR_BASE32..." }
  }]
}
```

### 3.2 Server-side middleware

**TypeScript / Express (the AgenticPlace primary):**

```ts
```

```
import express from "express";
import { paymentMiddleware, x402ResourceServer } from "@x402-avm/express";
import { registerExactAvmScheme } from "@x402-avm/avm/exact/server";
import { HTTPFacilitatorClient } from "@x402-avm/core/server";
import { ALGORAND_MAINNET_CAIP2, USDC_MAINNET_ASA_ID } from "@x402-avm/avm";

const facilitator = new HTTPFacilitatorClient({
  url: process.env.FACILITATOR_URL ?? "https://facilitator.goplausible.xyz",
});
const server = new x402ResourceServer(facilitator);
registerExactAvmScheme(server);

app.use(paymentMiddleware({
  "POST /v1/inference": {
    accepts: {
      scheme: "exact",
      network: ALGORAND_MAINNET_CAIP2,
      payTo: process.env.PAY_TO!,
      asset: USDC_MAINNET_ASA_ID,   // "31566704"
      price: "$0.01",
      maxTimeoutSeconds: 60,
    },
    description: "LLM inference per call",
    mimeType: "application/json",
  },
}, server));
```

**PHP / Laravel (for the existing PHP server side; Python port to follow):**

```
php
```

```php
class X402 {
    public function handle(Request $req, Closure $next, string $price = '10000') {
        $req_obj = [
          'scheme'=>'exact',
          'network'=>'algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=',
          'amount'=>$price, 'asset'=>'31566704',
          'payTo'=>config('x402.pay_to'),
          'maxTimeoutSeconds'=>60,
          'extra'=>['feePayer'=>config('x402.fee_payer')],
        ];
        $sigHdr = $req->header('PAYMENT-SIGNATURE') ?? $req->header('X-PAYMENT');
        if (!$sigHdr) {
            return response()->json(['x402Version'=>2,'accepts'=>[$req_obj]], 402)
              ->header('PAYMENT-REQUIRED',
                       base64_encode(json_encode(['x402Version'=>2,'accepts'=>[$req_
        }
        $payload = json_decode(base64_decode($sigHdr), true);
        $v = Http::post("{$this->facilitator}/verify",
            ['paymentPayload'=>$payload,'paymentRequirements'=>$req_obj])->json();
        if (!($v['isValid'] ?? false)) {
            return response()->json(['error'=>$v['invalidReason']], 402);
        }
        $resp = $next($req);
        $s = Http::post("{$this->facilitator}/settle",
            ['paymentPayload'=>$payload,'paymentRequirements'=>$req_obj])->json();
        $resp->headers->set('PAYMENT-RESPONSE', base64_encode(json_encode($s)));
        return $resp;
    }
}
```

**Python / FastAPI (the planned port path):** uses the official `x402-avm[fastapi]` package; one `add_middleware(PaymentMiddlewareASGI, …)` call replaces all hand-rolled wire logic. GitHub

### 3.3 Parsec client signing

Parsec exposes a `ClientAvmSigner` interface compatible with `@txnlab/use-wallet`, Pera, Defly, and Kibisis. The signing path is identical regardless of whether the seed lives in a Rust core or a TypeScript core: GitHub

```ts
```

```
async function buildPayment(client, req, parsec): Promise<PaymentPayload> {
  const sp = await client.getTransactionParams().do();
  const axfer = algosdk.makeAssetTransferTxnWithSuggestedParamsFromObject({
    sender: parsec.address, receiver: req.payTo,
    amount: BigInt(req.amount), assetIndex: Number(req.asset),
    suggestedParams: { ...sp, fee: req.extra?.feePayer ? 0 : 1000, flatFee: true },
  });
  if (!req.extra?.feePayer) {
    const sig = await parsec.sign(axfer.toByte());
    return { paymentIndex: 0, paymentGroup: [b64(sig)] };
  }
  const feePay = algosdk.makePaymentTxnWithSuggestedParamsFromObject({
    sender: req.extra.feePayer, receiver: req.extra.feePayer,
    amount: 0, suggestedParams: { ...sp, fee: 2000, flatFee: true },
  });
  algosdk.assignGroupID([feePay, axfer]);
  const signedAxfer = await parsec.sign(axfer.toByte());
  return {
    paymentIndex: 1,
    paymentGroup: [b64(feePay.toByte()), b64(signedAxfer)],
  };
}
```

### 3.4 Facilitator strategy and DELTAVERSE token support

For launch, point AgenticPlace at `facilitator.goplausible.xyz` (Algorand Foundation-endorsed, supports mainnet + testnet, Bazaar-compatible discovery). For sovereignty, deploy a forked `@x402-avm/core/facilitator` on AgenticPlace infra under `facilitator.agenticplace.com`, custodying a fee-payer hot wallet funded with ~10 ALGO and an auto-topup keeper. **DELTAVERSE-ecosystem token support** comes for free: any opted-in ASA ID can populate the `asset` field. The wallet auto-prompts for opt-in (0.1 ALGO MBR each) on first encounter with a new ASA-priced route.

### 3.5 Unresolved issues to lock down at launch

- **Idempotency on retry.** Cache `(payer, resource, last 60s)` server-side; replay the original `PAYMENT-RESPONSE` for duplicates. Without this, agents that retry on network errors will resign new axfers with later `lastValid` and double-pay.
- `upto` **scheme for per-token LLM billing is not yet specified for Algorand.** Either build a custom AVM `upto` scheme (escrow + reconciliation app) or use `exact` with conservative pre-quotes plus refund axfer. Default for v1: `exact` with refund.

- **Refunds.** No protocol primitive; implement business-logic refunds via a separate signed axfer from server → client.
- **Security.** Pin `@x402-avm/* ⩾ 2.6.1` (GHSA-qr2g-p6q7-w82m signature-bypass fix). Agentpaytrend

---

## 4. Algorand quantum-awareness via state proofs

Algorand State Proofs are the chain's post-quantum primitive: Falcon-1024 signatures (NIST FIPS 206 Level 5) over Merkle commitments to participation keys, produced every 256 rounds by the top 1024 online accounts, requiring 30 % signed weight to be valid. They went live on mainnet on **7 September 2022** with consensus v34, and the AVM gained the `falcon_verify` opcode in v4.3.0 (September 2024). Algorand + 5

### 4.1 What is and is not post-quantum

The split is essential and Parsec must surface it explicitly. **Account spend authority is Ed25519** and remains quantum-broken in principle by Shor; only opt-in Falcon logic-sig PQ accounts (since September 2024) are post-quantum on the spend path. **Block proposer signatures are Ed25519+VRF**; also Shor-breakable. **State proof participation is Falcon-1024 wrapped in a Merkle Signature Scheme** of ephemeral one-time keys; this is post-quantum. **Historical block integrity is anchored by Falcon every 256 rounds**, post-quantum once the SP is on chain. Algorand + 2

The honest framing: **even a future quantum adversary that breaks Ed25519 cannot rewrite the past once a Falcon state proof has been written for that interval.** Live spends still rely on Ed25519 unless the user has opted into a Falcon logic-sig account. Algorand Algorand

### 4.2 Verification cost reality check

A reference verification runs ~640 Falcon-1024 verifies (~158 k cycles each) plus ~640 SumHash512 Merkle paths plus ~640 SHA-256 transaction commitment paths. Native CPU: ~100 ms total, easily mobile. **EVM L1: not feasible end-to-end.** A pure-Solidity Falcon-512 verify is 1.8 M–7 M gas; Falcon-1024 is 2–3× heavier; 640 verifies pushes into 2–4 billion gas territory against a 30 M block limit. Even with the proposed EIP-8052 Falcon precompile (~3 k gas per verify), the SumHash512 Merkle paths have no precompile. Hacken Hacken

**Recommendation: do not build a from-scratch Solidity ASP verifier.** Use the Algorand Foundation's announced Groth16 SNARK pathway (the entire reason participants tree hashing was migrated to SumHash512 in PR #3895). Until the SNARK verifier ships, use **Wormhole-guardian attestation** as the liveness path with a Parsec UI second signal showing "PQ-anchored" only after the corresponding Algorand SP for the relevant interval lands. Medium

### 4.3 Parsec finality enum and UX

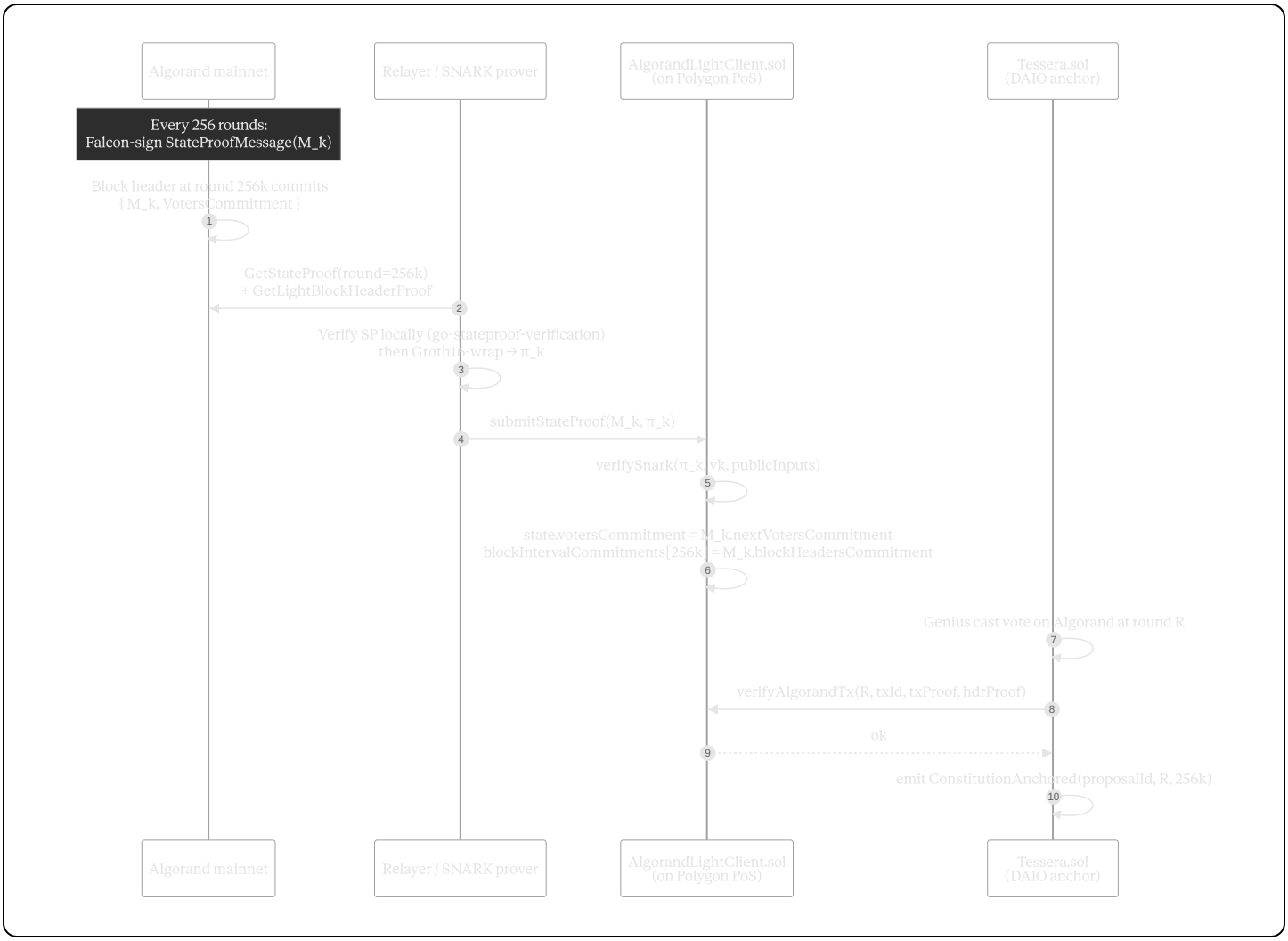
```ts
export enum TxFinality {
  PENDING            = "pending",
  CONSENSUS_FINAL    = "consensus_final",   // Algorand BA finalized; instant
  STATE_PROOF_FINAL  = "state_proof_final", // SP for this interval accepted on-chai
  CROSS_CHAIN_FINAL  = "cross_chain_final", // SP verified on the destination EVM li
}

export interface TxStatus {
  txId: string;
  round: number;
  finality: TxFinality;
  stateProofRound?: number;
  stateProofTxId?: string;
  evmAnchor?: { chainId: number; lightClientAddress: string; submittedAtBlock: numbe
  pqProtected: { historyByFalcon: boolean; accountByFalcon: boolean };
}
```

UX rules: balances and receipts get a green "PQ-anchored" badge only when finality ⩾ `STATE_PROOF_FINAL`. Outgoing high-value tx UX shows a delayed badge that flips green after the next SP. Incoming high-value tx warns if not yet `STATE_PROOF_FINAL`. Governance-weight votes that propagate cross-chain via Tessera require `STATE_PROOF_FINAL` before forwarding.

### 4.4 Bridging BANKON identity and DAIO governance to EVM

The canonical pattern for cross-chain anchoring of constitutional events:




```solidity
interface IAlgorandLightClient {
    function submitStateProof(StateProofMessage calldata m, bytes calldata snark) ex
    function verifyAlgorandTx(
        uint64 round,
        bytes32 txId,
        bytes32[] calldata txProofPath,
        bytes32[] calldata hdrProofPath
    ) external view returns (bool);
    function lastAnchoredRound() external view returns (uint64);
}
```

DAIO emits `ConstitutionAnchored(proposalId, algoRound, algoSpRound)` only after step 4 succeeds. This is the strongest "even after Q-Day" guarantee available today on EVM.

### 4.5 Open ARC opportunity

There is no Algorand ARC currently standardizing the wallet-level state-proof finality API. **Propose ARC-00xx: State-Proof Finality Wallet API** that adds the `TxStatus.finality` enum

above and a `getStateProofForRound(round)` call to `window.algorand`, so Parsec and other wallets surface PQ-finality consistently. This positions Parsec as the reference implementation. GitHub

---

## 5. HD wallet integration via ARC-52

### 5.1 What `go-algorand-hd-wallet` actually is

`AlgoNode/go-algorand-hd-wallet` is a Go port of the **ARC-52 / BIP32-Ed25519 (Khovratovich-Law)** standard, Apache-2.0, last updated April 5 2026. It is one of four sibling implementations alongside the Algorand Foundation's TS, Kotlin, and Swift references (`xHD-Wallet-API-{ts,kt,swift}`). All four implement the same derivation tree and signing surface; the Go port is the newest and least audited.

ARC-52 follows BIP-44 paths: `m / 44' / coin' / account' / change / keyIndex` where `coin = 283'` selects address keys and `coin = 0'` selects identity keys (`KeyContext.Address` vs `KeyContext.Identity`). The derivation algorithm is **not** SLIP-0010 Ed25519 (which is hardened-only); it is the Khovratovich-Law non-linear-keyspace scheme that allows soft public derivation at the lowest two levels. The Foundation libs default to the **Peikert** parameterization (9 bits of `kL` retained, 254 randomized) over the original Khovratovich (32 bits retained, 224 randomized); Peikert caps depth at ~8 levels but BIP-44's 5 fit easily. GitHub + 4

Critically, **the legacy 25-word Algorand mnemonic is incompatible with ARC-52**. It encodes a single 32-byte Ed25519 seed with no derivation tree. Users with 25-word phrases remain single-account; HD requires migration to a fresh BIP-39 24-word seed.

### 5.2 Parsec integration recommendation: port, do not wrap

Wrapping the Go library via TinyGo-WASM, cgo C-shared, or gomobile is feasible but lossy:

| Option | Pros | Cons |
|---|---|---|
| TinyGo → WASM | Smallest binary | TinyGo crypto stdlib incomplete; HMAC-SHA512 historically broken |
| cgo C-shared | Full Go stdlib | Multi-arch binary matrix (5 OS × 2 arch); Ed25519 secrets cross FFI |
| gomobile | Native iOS/Android works | RN bridge awkward; runtime overhead |
| **Native port** | **Audited libs available; <600 LOC** | **Re-implement the standard in TS or Rust** |

**Parsec should port the standard in its native language.** The Foundation's TS reference (`@algorandfoundation/xhd-wallet-api`) is Apache-2.0, deliberately migrated off WASM/LibSodium to Noble crypto for cross-platform RN support, and is the smallest path to feature parity. For a Rust core, port `xHD-Wallet-API-kt` against `ed25519-dalek` + `curve25519-dalek` + `hmac` + `sha2` + `pbkdf2` + `bip39`. Total surface is ~600 LOC plus tests. GitHub GitHub

```ts
// TypeScript path for Parsec
import { XHDWalletAPI, KeyContext, BIP32DerivationType }
  from "@algorandfoundation/xhd-wallet-api";
import { mnemonicToSeed, generateMnemonic } from "@scure/bip39";
import { wordlist } from "@scure/bip39/wordlists/english";
import algosdk from "algosdk";

const api = new XHDWalletAPI();
const mnemonic = generateMnemonic(wordlist, 256);
const seed = await mnemonicToSeed(mnemonic);
let rootKey = api.fromSeed(seed); seed.fill(0);

// Spending account: m/44'/283'/0'/0/0
const spendPk = await api.keyGen(rootKey, KeyContext.Address, 0, 0, 0);
const spendAddr = algosdk.encodeAddress(spendPk);

// Identity DID address: m/44'/0'/0'/0/0
const idPk = await api.keyGen(rootKey, KeyContext.Identity, 0, 0, 0);
const didAddr = algosdk.encodeAddress(idPk);

// Per-host x402 agent key: m/44'/283'/0xA402'/0/index
const agentPk = await api.keyGen(rootKey, KeyContext.Address, 0xA402, 0, 0);
rootKey.fill(0);
```

All Foundation HD libraries carry an explicit "NOT audited, NOT recommended for production" notice. **A security review is required before mainnet.** This is non-optional. GitHub

### 5.3 HD sub-accounts and soulbound BANKON identity

The right architectural seam is the ARC-52 split between `coin = 283'` (spending) and `coin = 0'` (identity). Concretely:

- **One BANKON AlgoIDNFT per master seed**, bound to the identity address derived at `m'/44'/0'/0'/0/0`. This is the user's DID address.

- **All spending sub-accounts live under** `m'/44'/283'/account'/change/index` and are not the NFT holder. They link to the identity by signed ARC-60 challenges from the identity key, stored in a "linkage" Algorand app box: `(identity_addr → set<sub_addrs>)`.
- **Per-host x402 agent keys** derive under a dedicated `account = 0xA402` with high `keyIndex` rotation, so machine-driven micropayments do not muddy the user's primary spending account's privacy or reputation.

Why one identity NFT per seed, not per derived account: identity is about the human or agent, not their financial compartments. Multiple NFTs would fragment Sybil-resistance. Discovery is preserved because any sub-account proves identity by signing a challenge with the identity key.

Hardware wallet caveat: Ledger and Keystone today implement SLIP-0010 Ed25519 (single account), not ARC-52. Until firmware updates ship, hardware accounts cannot mint AlgoIDNFTs directly; Parsec should detect this and either disable identity minting on hardware accounts or mint to a software-held identity key linked via signed cross-attestation.

---

## 6. Allchain mapping schema

The user-private `agenticplace.pythai.net/allchain.html` could not be fetched in this research session. The schema below is proposed *de novo*, EIP-3085 / CAIP-2 / ethereum-lists-aligned, with feature flags Parsec consumes at runtime to decide which capabilities to surface per chain.

### 6.1 JSON Schema

```
json
```

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://agenticplace.pythai.net/allchain.schema.json",
  "type": "object",
  "required": ["version", "updated", "chains"],
  "properties": {
    "version": { "type": "string" },
    "updated": { "type": "string", "format": "date-time" },
    "chains": { "type": "array", "items": { "$ref": "#/$defs/Chain" } }
  },
  "$defs": {
    "Chain": {
      "type": "object",
      "required": ["id","caip2","name","family","rpc","nativeCurrency"],
      "properties": {
        "id":      { "type": "string" },
        "caip2":   { "type": "string" },
        "chainId": { "type": ["integer","string","null"] },
        "name":    { "type": "string" },
        "family":  { "enum": ["evm","algorand","bitcoin","solana","cosmos","other"]
        "network": { "enum": ["mainnet","testnet","devnet","localnet"] },
        "rpc":     { "type": "array", "items": { "type": "string" } },
        "indexer": { "type": "string" },
        "explorer":{ "type": "array" },
        "nativeCurrency": { "type": "object",
          "properties":{"name":{},"symbol":{},"decimals":{"type":"integer"}}},
        "gasToken": { "type": "string" },
        "bridge":   { "type": "object",
          "properties":{
            "kind":{"enum":["agglayer-lxly","cctp","wormhole","state-proof","oracle"
            "contract":{"type":"string"},
            "destinations":{"type":"array","items":{"type":"string"}}}},
        "features": { "type": "array", "items": { "enum":[
          "EIP155","EIP1559","EIP4844","ERC4337",
          "ARC4","ARC56","ARC72","StateProofs","Boxes","InnerTxn",
          "Ordinals","BRC20","Runes",
          "AggLayer","CDK","Validium","Multisig","BFT",
          "SubSecondFinality","x402","Paymaster","BANKON","DAIO"
        ]}},
        "contracts": { "type": "object", "additionalProperties": { "type": "string"
        "x402":     { "type": "object",
          "properties": {
            "supported": { "type": "boolean" },
```

```
            "facilitator": { "type": "string" },
            "asset": { "type": "string" }}},
        "trust":    { "type": "object",
          "properties":{
            "model":{"enum":["pos","ppos","bft","zk-rollup","optimistic","validium",
            "validators":{"type":"integer"},"quorum":{"type":"string"},"admin":{"typ
        "parsec":   { "type": "object",
          "properties":{
            "enabled":{"type":"boolean"},"default":{"type":"boolean"},
            "signing":{"enum":["secp256k1","ed25519","schnorr"]},
            "addressFormat":{"enum":["evm-hex","algorand-base32","bech32","bech32m",
            "constitutionalLayer":{"type":"boolean"},
            "identityLayer":{"type":"boolean"},
            "auditMirror":{"type":"boolean"}}}
      }
    }
  }
}
```

### 6.2 Concrete entries

```
json
```

```
{
  "version": "1.0.0",
  "updated": "2026-04-30T00:00:00Z",
  "chains": [
    {
      "id":"polygon-pos-mainnet","caip2":"eip155:137","chainId":137,
      "name":"Polygon PoS","family":"evm","network":"mainnet",
      "rpc":["https://polygon-rpc.com"],
      "explorer":[{"name":"polygonscan","url":"https://polygonscan.com"}],
      "nativeCurrency":{"name":"POL","symbol":"POL","decimals":18},
      "features":["EIP155","EIP1559","ERC4337","DAIO"],
      "contracts":{"Constitution":"0x...","Genius":"0x...","Tabularium":"0x..."},
      "x402":{"supported":true,"facilitator":"https://api.cdp.coinbase.com","asset"
      "trust":{"model":"pos","validators":110,"quorum":"2/3-stake"},
      "parsec":{"enabled":true,"default":true,"signing":"secp256k1",
                "addressFormat":"evm-hex","constitutionalLayer":true}
    },
    {
      "id":"algorand-mainnet",
      "caip2":"algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=",
      "chainId":"mainnet-v1.0","family":"algorand","network":"mainnet",
      "rpc":["https://mainnet-api.algonode.cloud"],
      "indexer":"https://mainnet-idx.algonode.cloud",
      "nativeCurrency":{"name":"Algorand","symbol":"ALGO","decimals":6},
      "features":["ARC4","ARC56","ARC72","StateProofs","Boxes","InnerTxn",
                  "SubSecondFinality","x402","BANKON"],
      "contracts":{"BANKON_AlgoIDNFT":"<APP_ID>","DID_Storage":"<APP_ID>",
                   "BondNFT":"<APP_ID>"},
      "x402":{"supported":true,"facilitator":"https://facilitator.goplausible.xyz","
      "bridge":{"kind":"state-proof","destinations":["polygon-pos-mainnet","ethereum
      "trust":{"model":"ppos","quorum":"2/3-stake"},
      "parsec":{"enabled":true,"default":false,"signing":"ed25519",
                "addressFormat":"algorand-base32","identityLayer":true}
    },
    {
      "id":"openbdk-bdk6","caip2":"eip155:60606","chainId":60606,
      "name":"openBDK BDK6 (1+3 BFT)","family":"evm","network":"mainnet",
      "rpc":["https://rpc.bdk6.local"],
      "nativeCurrency":{"name":"BDK","symbol":"BDK","decimals":18},
      "features":["EIP155","BFT"],
      "trust":{"model":"bft","validators":3,"quorum":"2-of-3"},
      "parsec":{"enabled":true,"default":false,"signing":"secp256k1",
                "addressFormat":"evm-hex","auditMirror":true}
```

```
    },
    {
      "id":"arc-testnet","caip2":"eip155:5042002","chainId":5042002,
      "name":"Arc Testnet (Circle)","family":"evm","network":"testnet",
      "rpc":["https://rpc.testnet.arc.network"],
      "nativeCurrency":{"name":"USD Coin","symbol":"USDC","decimals":6},
      "gasToken":"USDC",
      "features":["EIP155","SubSecondFinality","x402","Paymaster"],
      "trust":{"model":"bft","admin":"Circle"}
    },
    {
      "id":"bitcoin-mainnet","caip2":"bip122:000000000019d6689c085ae165831e93",
      "chainId":null,"name":"Bitcoin Mainnet","family":"bitcoin","network":"mainnet"
      "rpc":["https://bitcoin-rpc.publicnode.com"],
      "nativeCurrency":{"name":"Bitcoin","symbol":"BTC","decimals":8},
      "features":["Ordinals","BRC20","Runes"],
      "trust":{"model":"l1-pow"}
    }
  ]
}
```


### 6.3 Runtime consumption pattern in Parsec

```ts
```

```
// parsec/src/registry.ts
import schema from "./allchain.schema.json";
const allchain: AllChain = await fetch("/allchain.json")
  .then(r => r.json())
  .then(validateAgainst(schema));

// Feature gates everywhere in the wallet:
function canPay(chain: ChainEntry): boolean {
  return chain.x402?.supported === true;
}
function isConstitutionalChain(chain: ChainEntry): boolean {
  return chain.parsec?.constitutionalLayer === true
      && chain.contracts?.Constitution != null;
}
function isIdentityChain(chain: ChainEntry): boolean {
  return chain.parsec?.identityLayer === true
      && chain.contracts?.BANKON_AlgoIDNFT != null;
}
function pqAnchorTarget(chain: ChainEntry): boolean {
  return chain.features.includes("StateProofs");
}
```

The wallet UI uses these gates to decide which chains to default to for a given action: a payment defaults to the user's primary `default: true` chain; an identity flow forces the chain with `identityLayer: true`; a governance vote forces `constitutionalLayer: true`.

---

## 7. Foundry + AlgoKit testing and deployment workflow

### 7.1 Foundry project layout

```
daio/
├── foundry.toml                # solc 0.8.28, optimizer 10000, viaIR,
bytecode_hash="none"
├── remappings.txt
├── lib/{forge-std,openzeppelin-contracts,openzeppelin-contracts-upgradeable}
├── src/
│   ├── core/{Constitution,RoleRegistry,Treasury}.sol
│   ├── governance/{Genius,BonaToken,Senatus,Censura}.sol
│   ├── reputation/{Tabularium,Fides,SponsioPactum}.sol
│   ├── crosschain/Tessera.sol
│   └── interfaces/
```

```
├── script/
│   ├── Deploy.s.sol
│   └── PostDeployWiring.s.sol
└── test/
    ├── unit/
    ├── integration/
    ├── invariant/{handlers/,ConstitutionalInvariants.t.sol}
    └── fork/
```

Key `foundry.toml` settings: `solc = "0.8.28"`, `optimizer_runs = 10_000`, `via_ir = true`, `bytecode_hash = "none"` for stable CREATE2 across CI runs and chains, `[fuzz] runs = 10_000`, `[invariant] runs = 512, depth = 64, fail_on_revert = false, show_metrics = true`.

### 7.2 Test pyramid

Unit tests (per-contract behavior) run in milliseconds against `revm`. Fuzz tests (10k runs of parameter ranges for the reputation accumulator and decay function) catch numerical edge cases. Invariant tests against a handler contract enforce the constitution: `invariant_NoRoleConflict`, `invariant_TotalReputationBounded`, `invariant_TreasuryConservation`. Fork tests (`forge test --fork-url $POLYGON_RPC`) validate interaction with deployed Safes, Chainlink feeds, and cross-chain Tessera anchor. Medium

### 7.3 AlgoKit parallel project

```
algoidnft/
├── .algokit.toml                # min_version v3.0.0
├── pyproject.toml               # algorand-python ^3.1.1, algokit-utils ^3
├── smart_contracts/
│   ├── algo_id_nft/contract.py
│   ├── did_storage/contract.py
│   ├── bond/contract.py
│   └── linkage/contract.py
├── tests/
│   ├── unit/                    # algopy_testing_context() — offline,
milliseconds
│   ├── integration/             # against algokit localnet — seconds
│   └── e2e/                     # against testnet — minutes
└── .env.{localnet,testnet,mainnet}
```

Critical reality check: **Beaker is deprecated** (the `algokit-beaker-default-template` carries an explicit deprecation banner; CTO John Woods pre-announced this with AlgoKit 2.0 in December 2023). New work goes to **Algopy** (PuyaPy compiler, `algorand-python ⩾ 3.1.1`, October 2025). PyTeal remains in maintenance mode but is no longer recommended. GitHub + 3

### 7.4 Foundry vs AlgoKit — what is missing and how to compensate

| Capability | Foundry | AlgoKit | Compensation |
|---|---|---|---|
| In-process VM unit tests | revm, ms | algopy_testing, ms (offline) | parity on unit tier |
| Stateful fuzzing / invariants | first-class | not built in | use Hypothesis + algopy_testing manually |
| Forking mainnet | `--fork-url` instant | none | replay against localnet/testnet |
| Source verification | Etherscan/Sourcify/Blockscout | no Etherscan equivalent | upload ARC-56 spec to Lora; bytecode hash + source upload to AlgoExplorer |
| Full-suite speed | sub-second to seconds | unit: seconds; integration: minutes | budget 10–50× longer CI on the Algorand side |

The Algorand side has no `revm` equivalent for full integration tests. Plan accordingly: keep unit tests offline via `algopy_testing_context()`, accept slower integration runs against LocalNet, and reserve testnet runs for E2E rehearsals before mainnet. GitHub

### 7.5 Mainnet deployment runbook

**Pre-deploy (T-7d to T-0):** All contracts pinned to a tagged commit; `forge build --sizes` shows every contract under 24 KB; `forge test -vv` green; coverage ⩾ 90 %; invariant suite green at `runs ⩾ 1024, depth ⩾ 128` overnight; Slither + Aderyn run; external audit complete with remediation re-tested; storage layout snapshotted for Tabularium UUPS proxy; deployer EOA loaded into a dedicated Safe; multisig signers (4-of-7) rehearsed; Timelock minimum delay set to 48 h; CREATE2 salts pre-computed and predicted addresses cross-checked across all target chains; on the Algorand side `algokit project deploy testnet` rehearsed and AlgoIDNFT app-id reserved.

**Deploy sequence (T-0):** Algorand side first — `algokit project deploy mainnet --ci` stands up AlgoIDNFT (ARC-72 soulbound) and the did:algo storage app, capturing app-IDs. EVM constitutional layer next — `forge script script/Deploy.s.sol --broadcast --slow --verify --resume` deploys the 9 contracts in dependency order. Verify each broadcast against `broadcast/Deploy.s.sol/137/run-latest.json`; if any tx is unconfirmed > 5 min, use `--resume`, never blind retry. Run `PostDeployWiring.s.sol` to grant roles, set DIDAnchor → AlgoIDNFT app-id mapping, and transfer ownership to the Timelock. Foundry

**Verification (T+0):** Bytecode of every deployed contract sha256-matches `out/`; Etherscan/Sourcify/Blockscout shows verified source for all 9 contracts (full match preferred); `Initialized(uint64)` events present for UUPS proxies; `roles.owner() == address(timelock)`; deployer EOA holds no privileged roles; AlgoIDNFT `arc72_transferFrom` reverts on testnet smoke call; box for issuer DID populated; `algoid retrieve did:algo:mainnet:app:<APP_ID>:<SUBJECT>` returns expected document.

**Post-deploy (T+0 to T+7d):** Block-explorer alerts on `Upgraded`, `RoleGranted`, `OwnershipTransferred` via Tenderly/OpenZeppelin Defender/Forta; pause/emergency mechanism tested on a fork; disaster-recovery runbook published (signers' key locations, hardware seed rotation schedule); public deployment artifact published (addresses, ABIs, constructor args, bytecode hashes, commit SHA); DID ↔ Bond NFT cross-chain anchoring tested by minting an AlgoIDNFT then calling DIDAnchor.set on EVM; 7-day observation window before granting any DAO control over upgrades.

**Pitfalls to avoid:** CREATE address depends on deployer nonce (use CREATE2 with stable salts, pin solc, set `bytecode_hash="none"`, reserve addresses promptly to prevent squatting); UUPS initialization races (always deploy proxy + call `initialize` atomically via constructor); owner-transfer races (use `Ownable2Step` or transfer directly to Timelock in deploy tx); RPC verifier flakiness on load-balanced endpoints (use the chain's official single-endpoint public RPC, not aggregators). Readthedocs

---

## Deployment-sequence roadmap

The end-to-end rollout sequences as follows. **Phase 0 (foundations, weeks 0–4)** stands up the BANKON AlgoIDNFT ARC-72 soulbound contract, the did:algo storage app, AlgoKit project skeleton, and the Foundry project skeleton with the 9 contract stubs and invariant harness. **Phase 1 (identity + payments, weeks 4–10)** ships AlgoIDNFT to Algorand testnet, integrates ARC-52 HD derivation into Parsec (port path, not FFI), wires x402 v2 with `@x402-avm/express` middleware on AgenticPlace pointed at `facilitator.goplausible.xyz`, and lands the mindX session-JWT auth flow against AlgoIDNFT. **Phase 2 (governance, weeks 10–20)** completes the 9-contract Solidity suite, runs the invariant suite overnight at `runs=1024,depth=128`, completes external audit + remediation, deploys to Polygon Amoy testnet, runs the genesis-ceremony

rehearsal with founders. **Phase 3 (mainnet, weeks 20–24)** runs the deployment runbook in §7.5: AlgoIDNFT to Algorand mainnet first, then DAIO/BONAFIDE to Polygon PoS, then Tessera cross-chain anchor, then handover from deployer EOA to Senatus + Timelock. **Phase 4 (post-quantum anchor, weeks 24–32)** integrates the state-proof finality enum into Parsec UI, deploys the AlgorandLightClient.sol (initially with a Wormhole-guardian liveness path until the Foundation's Groth16 SNARK ships), and starts emitting `ConstitutionAnchored` events for governance-weight votes. **Phase 5 (replication and AggLayer, weeks 32–40)** stands up the BDK6 audit replica receiving every constitutional event and, once AggLayer v0.4 fast interop ships, mirrors the constitutional contracts to a CDK chain (Katana or self-deployed via Gateway) for cross-aggchain composability.

The key path ordering: identity must precede payments (so paid calls can carry verified DIDs), payments must precede governance (so citizen onboarding can charge an x402 fee for Sybil resistance), governance must precede cross-chain anchoring (so there are constitutional events worth anchoring), and post-quantum anchoring must precede replication (so BDK6 mirrors PQ-finalized state, not transient state).

## Conclusion

The non-obvious decisions in this design come from rejecting tempting symmetries. The first is **not porting the Solidity suite to Algopy** despite identity already being on Algorand; the 6–12-week rewrite cost, loss of audit equivalence, and the 8-entry reference-list ceiling of Algorand inter-contract calls outweigh any composability win, while a one-way state-proof bridge gives you the cross-chain semantics you actually need. The second is **not building a Solidity Falcon-1024 verifier from scratch**; verifying 640 Falcon signatures plus SumHash512 Merkle paths costs 2–4 billion gas, which is exactly why the Algorand Foundation is shipping a Groth16 SNARK pathway and exactly why DAIO should consume that pathway when it lands rather than racing it. The third is **porting ARC-52 into Parsec's native language rather than wrapping AlgoNode's Go library via FFI/WASM**; the cryptographic surface is ~600 LOC and the Foundation's TS reference is already designed for cross-platform RN. The fourth is **deploying to Polygon PoS, not Polygon CDK or zkEVM**; CDK adds Polygon's 2-of-3 + 6-of-8 multisig assumption on top of your sequencer responsibility, and zkEVM is sunsetting in 2026.

The big architectural insight is that **post-quantum security in this stack is not about Falcon-signing every transaction — it is about anchoring the *history* of governance and identity events to Falcon-signed state proofs every 256 rounds, while letting day-to-day spends remain on Ed25519 until users opt into Falcon logic-sig accounts**. That reframing turns "quantum-aware wallet" from a marketing slogan into an engineering primitive: a four-level finality enum that flips green when a state proof for the relevant interval lands, and a Tessera contract that emits `ConstitutionAnchored` only after that proof has been verified on the destination EVM light client. This is the strongest "even after Q-Day" guarantee currently shippable, and it is uniquely available because Algorand has been signing with Falcon-1024 in

production since September 2022 and shipped `falcon_verify` to the AVM in September 2024 — well ahead of any EVM PQ precompile. Algorand + 2

The integration spine across AgenticPlace, mindX, BANKON, DAIO, Parsec, and BDK6 is therefore not a federation of equals; it is a **layered trust stack** where each layer answers a single question. AgenticPlace answers "who is making this call and have they paid?" via DID-auth + x402. mindX answers "what is the gamma-training output?" and trusts only the session JWT. BANKON answers "is this DID still active and what does it claim?" via the ARC-72 soulbound NFT and did:algo registry. DAIO answers "what does the constitution say about this action?" via immutable Solidity contracts on Polygon PoS. Parsec answers "is this transaction post-quantum anchored yet?" via the four-level finality enum. BDK6 answers "what is the audit log of every constitutional event?" via append-only replication. Each component does one thing, the seams between them are signed, and the whole stack degrades gracefully — a mindX outage does not affect governance, an Algorand state-proof delay does not block live payments, a BDK6 validator failure does not impair anything user-visible.